Welcome to Code Crunch!

C1|Crunch Convos

WED 2-3PM Weekly | Spring 2025

Attendance

Earn a Certificate of Completion by attending at least 70% of the workshops

linktr.ee/CODE.CRUNCH

Unit #5

C1 | Stacks & Queues

Stacks

A **stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle.

- The last element added to the stack is the first one to be removed.

Stack Operations:

- **Push**: Add an element to the top of the stack.
- **Pop**: Remove and return the top element from the stack.
- **Peek**: View the top element without removing it.
- **isEmpty**: Check if the stack is empty.

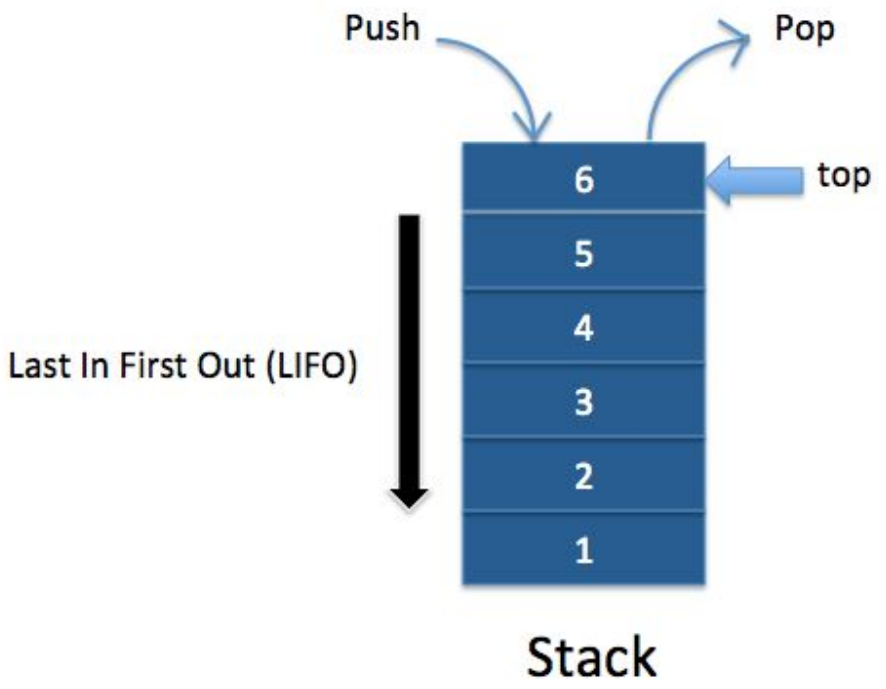


Image Source: Medium

Stacks in Python

In Python, we can easily implement a stack using a built-in **list**. Python lists provide all the necessary operations to simulate a stack, such as `append()` and `pop()`.

Key Operations with List:

- **Push**: Use `append()` to add an element to the end of the list (top of the stack).
- **Pop**: Use `pop()` to remove and return the last element (top of the stack).
- **Peek**: Access the last element of the list using `[-1]` without removing it.
- **isEmpty**: Check if the list is empty by comparing its length to `0`.

```
stack = []

stack.append(5)
stack.append(10)
stack.append(15)

print(stack[-1])  # 15

stack.pop() # 15

print(stack[-1])  # 10

is_empty = len(stack) == 0
print(is_empty)  # False

my_stack.pop()  # Removes 10
my_stack.pop()  # Removes 5

is_empty = len(stack) == 0
print(is_empty)  # True
```

Stack Fundamentals

Space Complexity: `O(n)` where n is the number of elements in the stack.

Time Complexity:

- **Push**: `O(1)` (adding an element to the top).
- **Pop**: `O(1)` (removing the top element).
- **Peek**: `O(1)` (viewing the top element).
- **isEmpty**: `O(1)` (checking if the stack is empty).

- **Advantages**:
  - Simple and efficient for **LIFO (Last In, First Out)** operations.
  - Great for problems like **backtracking** (e.g., undo/redo, navigating browser history).

- **Disadvantages**:
  - No random access to elements; only the top element can be accessed directly.
  - Not suitable for searching as elements can only be accessed in **LIFO** order.

Queues

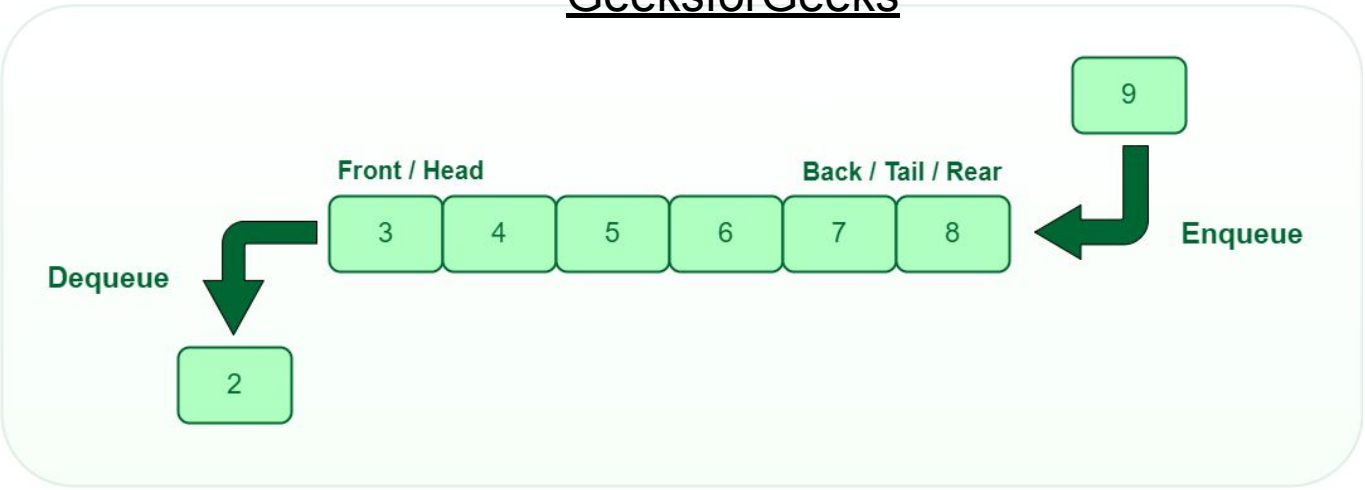
A **queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle.

- The first element added to the queue is the first one to be removed.

Queue Operations:

- **Enqueue**: Add an element to the back of the queue.
- **Dequeue**: Remove and return the front element of the queue.
- **Peek**: View the front element without removing it.
- **isEmpty**: Check if the queue is empty.

Image Source:
GeeksforGeeks



deque in Python

`deque` (double-ended queue) is a part of the `collections` module in Python and is optimized for fast appends and pops from both ends of the queue.

Why Use deque?:

- **Efficient Operations**: `deque` provides `O(1)` time complexity for appending and popping elements from both ends, unlike lists, which have `O(n)` complexity when popping from the front.

- **Versatility**: Unlike regular queues or stacks, deque can be used for both **LIFO** and **FIFO** operations.

deque Example

Common Methods in deque:

- `append(x)`: Adds x to the right end.
- `appendleft(x)`: Adds x to the left end.
- `pop()`: Removes and returns an element from the right end.
- `popleft()`: Removes and returns an element from the left end.

```
from collections import deque

# Create a deque
d = deque([1, 2, 3])

# Append and pop elements
d.append(4)          # [1, 2, 3, 4]
d.appendleft(0)      # [0, 1, 2, 3, 4]
d.pop()              # [0, 1, 2, 3]
d.popleft()          # [1, 2, 3]
```

Queues in Python

Using `deque` for queues is a great choice since it provides efficient `O(1)` operations for adding (enqueue) and removing (dequeue) elements from both ends.

Unlike lists, where `pop(0)` is `O(n)`, `deque.popleft()` is `O(1)`, making deque ideal for implementing queues.

```
from collections import deque

q = deque()
q.append(10)  # [10]
q.append(20)  # [10, 20]
q.append(30)  # [10, 20, 30]

# Remove from the front
q.popleft()  # Removes 10 -> [20, 30]

# Peek at the front
print(q[0])  # 20

# Check if queue is empty
is_empty = len(q) == 0
print(is_empty)  # False
```

Queue Fundamentals

Space Complexity: `O(n)` where n is the number of elements in the queue.

Time Complexity:

- **Enqueue**: `O(1)` (adding an element to the back).
- **Dequeue**: `O(1)` (removing the front element).
- **Peek**: `O(1)` (viewing the front element).
- **isEmpty**: `O(1)` (checking if the queue is empty).

- **Advantages**:
  - Ideal for **FIFO** tasks like **task scheduling**, and **breadth-first search (BFS)**.
  - Efficient for handling elements in the order they arrive.

- **Disadvantages**:
  - No random access to elements; can only operate on the front and back.

Practice Problem (Leetcode #20)

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Every close bracket has a corresponding open bracket of the same type.

Example
Input: s = "([ ])"
Output: True

Practice Problem (Leetcode #20)

Solution:

https://colab.research.google.com/drive/1CKuwQ_CaYxJh6wlrAKkyE1dCrWltA-O3?usp=sharing

Attendance

linktr.ee/CODE.CRUNCH

Join us for the next sessions

Crunch Convos: WED 2:30PM - 3:30PM
Crunch Code: WED 3:30PM - 4:30PM

RSVP lu.ma/CODECRUNCH

Your feedback helps us improve. Share your thoughts!
Go to our website: go.fiu.edu/codecrunch